



Institut Teknologi Bandung

Direktorat Pendidikan Profesional Berkelanjutan

x

CHAPTER 7

Menyelam ke Keras

Tiga cara membangun model, tiga tingkat kendali atas pelatihannya – dan satu asas yang menyatukan semuanya: mudah dimulai, tanpa langit-langit.

Durasi 3 jam (2 sesi)

Pengajar Rahman Indra Kesuma, S.Kom., M.Cs.

Sumber Chollet & Watson, Deep Learning with Python 3e – bab 7 (hlm. 190-230)

Tujuan Sesi

Setelah sesi ini, peserta dapat:

- 1 Menjelaskan asas **progressive disclosure of complexity** dan menempatkan diri sendiri pada spektrum alur kerja Keras.
- 2 Membangun model bermasukan-jamak dan berkeluaran-jamak dengan **Functional API**, lalu melatihnya dengan senarai maupun kamus.
- 3 Memanfaatkan **akses ke keterhubungan lapis**: menggambar topologi dengan `plot_model()` dan **mengekstraksi fitur** dari simpul antara.
- 4 Menulis **Model subclass** – dan menyebut apa yang hilang saat memilihnya.
- 5 Menulis **metrik kustom** (`update_state`, `result`, `reset_state`) dan **callback kustom**, serta memakai `EarlyStopping` + `ModelCheckpoint`.
- 6 Menulis **`train_step()` sendiri** di TensorFlow, PyTorch, dan JAX – dan menyisipkannya ke `fit()` supaya callback dan optimasi bawaan tetap terpakai.

BUKU

[Bab 7 — teks penuh](#)

NOTEBOOK

[01_tiga_cara_membangun_model.ipynb](#)

NOTEBOOK

[02_metrik_dan_callback_kustom.ipynb](#)

NOTEBOOK

[03_train_step_kustom_per_backend.ipynb](#)

REPO

[Notebook resmi penulis](#)

Progressive disclosure of complexity



Gambar 7.1 – bukan empat kerangka berbeda, melainkan satu spektrum di atas API yang sama.

Anda bisa memakai Keras seperti memakai scikit-learn – tinggal memanggil `fit()` dan membiarkan kerangkanya bekerja – atau memakainya seperti NumPy, dengan kendali penuh atas setiap detail kecil.

Chollet & Watson, bab 7.1

Analogi yang dipakai buku: **Keras adalah Python-nya deep learning**. Python multiparadigma – berorientasi objek, fungsional, prosedural, semuanya rukun. Karena itu **Anda tidak perlu pindah kerangka** saat berpindah dari mahasiswa ke peneliti, atau dari data scientist ke deep learning engineer.

01

Tiga cara membangun model

Sequential, Functional, subclassing – dan kapan masing-masing.

Sequential itu pada dasarnya senarai Python

Sequential – API yang paling mudah didekati. Batasnya juga jelas: ia hanya bisa menyatakan model dengan **satu masukan dan satu keluaran**, satu lapis sesudah lapis lain.

Padahal di praktik, model dengan **masukan jamak** (citra dan metadatanya), **keluaran jamak** (beberapa hal yang mau diramalkan), atau **topologi tak-linear** itu **biasa sekali**

Listing 7.8 – versi Functional

PYTHON

```
inputs = keras.Input(shape=(3,),
                       name="my_input")
features = layers.Dense(
    64, activation="relu")(inputs)
outputs = layers.Dense(
    10, activation="softmax")(features)

model = keras.Model(
    inputs=inputs, outputs=outputs,
    name="my_functional_model")
```

tensor simbolik

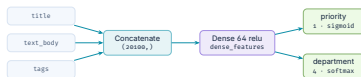
PYTHON

```
inputs.shape      # (None, 3) <- None = ukuran batch, bebas
inputs.dtype      # "float32"

features = layers.Dense(64, activation="relu")(inputs)
features.shape    # (None, 64)
```

`inputs` itu **tensor simbolik**: ia **tidak memuat data apa pun**, tetapi menyandikan spesifikasi tensor yang kelak akan dilihat model. Semua lapis Keras bisa dipanggil baik pada tensor data sesungguhnya maupun pada tensor simbolik – yang kedua mengembalikan tensor simbolik baru dengan shape dan dtype yang sudah diperbarui.

Di sinilah Functional API bersinar



Tiga masukan, dua keluaran, satu simpul antara yang dipakai bersama — tak mungkin ditulis sebagai Sequential.

Tiga masukan (judul, badan teks, tag), dua keluaran (skor prioritas dan departemen).

listing 7.9

PYTHON

```

vocabulary_size, num_tags, num_departments = 10000, 100, 4

title = keras.Input(shape=(vocabulary_size,), name="title")
text_body = keras.Input(shape=(vocabulary_size,), name="text_body")
tags = keras.Input(shape=(num_tags,), name="tags")

features = layers.Concatenate([title, text_body, tags])
features = layers.Dense(64, activation="relu", name="dense_features")(features)

priority = layers.Dense(1, activation="sigmoid", name="priority")(features)
department = layers.Dense(num_departments, activation="softmax",
                          name="department")(features)

model = keras.Model(inputs=[title, text_body, tags],
                    outputs=[priority, department])
  
```

Buku menyebutnya **seperti bermain LEGO**: cara yang sederhana tetapi sangat luwes untuk mendefinisikan graf lapis yang sebarang.

Melatihnya: senarai berurutan, atau kamus bernama

listing 7.10 – senarai

PYTHON

```
model.compile(
    optimizer="adam",
    loss=["mean_squared_error",
          "sparse_categorical_crossentropy"],
    metrics=[["mean_absolute_error"],
              ["accuracy"]])

model.fit(
    [title_data, text_body_data, tags_data],
    [priority_data, department_data],
    epochs=1)
```

listing 7.11 – kamus

PYTHON

```
model.compile(
    optimizer="adam",
    loss={"priority": "mean_squared_error",
          "department":
            "sparse_categorical_crossentropy"},
    metrics={"priority": ["mean_absolute_error"],
             "department": ["accuracy"]})

model.fit(
    {"title": title_data,
     "text_body": text_body_data,
     "tags": tags_data},
    {"priority": priority_data,
     "department": department_data},
    epochs=1)
```

Versi senarai **harus mengikuti urutan** yang Anda berikan ke konstruktor `Model()`. Kalau masukan atau keluarannya banyak, **pakai kamus bernama** – urutannya jadi tidak penting lagi, dan kodenya jauh lebih tahan salah.

Kekuatan sesungguhnya: akses ke keterhubungan lapis

menggambar topologi

PYTHON

```
keras.utils.plot_model(
    model, "ticket_classifier.png")

# versi yang jauh lebih menolong
# saat mengawakutu:
keras.utils.plot_model(
    model,
    "ticket_classifier_with_shape_info.png",
    show_shapes=True,
    show_layer_names=True)
```

listing 7.12 – memeriksa simpul

PYTHON

```
model.layers
model.layers[3].input
model.layers[3].output
# <KerasTensor shape=(None, 20100),
# dtype=float32>
```

listing 7.13 – ekstraksi fitur

PYTHON

```
# Mau menambah keluaran ketiga: taksiran kesulitan tiket.
# TIDAK perlu membangun dan melatih ulang dari nol.
features = model.layers[4].output          # lapis Dense antara tadi
difficulty = layers.Dense(3, activation="softmax", name="difficulty")(features)

new_model = keras.Model(
    inputs=[title, text_body, tags],
    outputs=[priority, department, difficulty])
```

`None` pada shape tensor menyatakan **ukuran batch** – model ini menerima batch berukuran berapa pun.

Model subclassing: kendali penuh

listing 7.14

PYTHON

```
class CustomerTicketModel(keras.Model):
    def __init__(self, num_departments):
        super().__init__()
        self.concat_layer = layers.Concatenate()
        self.mixing_layer = layers.Dense(64, activation="relu")
        self.priority_scorer = layers.Dense(1, activation="sigmoid")
        self.department_classifier = layers.Dense(num_departments,
                                                  activation="softmax")

    def call(self, inputs):
        # lintasan maju ditulis di sini
        features = self.concat_layer(
            [inputs["title"], inputs["text_body"], inputs["tags"]])
        features = self.mixing_layer(features)
        return self.priority_scorer(features), self.department_classifier(features)
```

	Layer	Model
Perannya	Bata bangunan untuk membuat model.	Objek tingkat atas yang benar-benar dilatih dan diekspor.
<code>fit()</code> , <code>evaluate()</code> , <code>predict()</code>	Tidak punya.	Punya.
Disimpan ke berkas	Tidak.	Bisa.

Selebihnya kedua kelas itu **praktis identik**. Subclassing membuka model yang **tidak bisa dinyatakan sebagai graf berarah tanpa siklus** – misalnya `call()` yang memakai lapis di dalam gelung `for`, atau bahkan memanggilnya secara rekursif.

Harga kebebasan itu

Yang hilang saat subclassing

`summary()` tidak menampilkan keterhubungan lapis ·
`plot_model()` tidak bisa dipakai · ekstraksi fitur tidak mungkin – sebab memang tidak ada grafnya.

Model Functional = struktur data eksplisit – graf lapis yang bisa Anda lihat, periksa, dan ubah.

Model subclass = **sepotong bytecode** – kelas Python dengan metode `call()` berisi kode mentah. Itulah sumber keluwesannya, dan sekaligus sumber batasannya.

Begitu model subclass diinstansiasi, **lintasan majunya menjadi kotak hitam sepenuhnya**. Anda mengembangkan objek Python baru, bukan sekadar menjepretkan bata LEGO – **permukaan galatnya jauh lebih luas, dan pekerjaan awakutunya lebih banyak**.

Ketiganya bisa dicampur – dan mana yang dipilih

listing 7.15 – subclass di dalam Functional

PYTHON

```
class Classifier(keras.Model):
    def __init__(self, num_classes=2):
        super().__init__()
        units, act = ((1, "sigmoid") if num_classes == 2
                      else (num_classes, "softmax"))
        self.dense = layers.Dense(units, activation=act)

    def call(self, inputs):
        return self.dense(inputs)

inputs = keras.Input(shape=(3,))
features = layers.Dense(64, activation="relu")(inputs)
outputs = Classifier(num_classes=10)(features)
model = keras.Model(inputs=inputs, outputs=outputs)
```

listing 7.16 – Functional di dalam subclass

PYTHON

```
inputs = keras.Input(shape=(64,))
outputs = layers.Dense(1, activation="sigmoid")(inputs)
binary_classifier = keras.Model(inputs, outputs)

class MyModel(keras.Model):
    def __init__(self):
        super().__init__()
        self.dense = layers.Dense(64, activation="relu")
        self.classifier = binary_classifier

    def call(self, inputs):
        return self.classifier(self.dense(inputs))
```

Saran buku: kalau model Anda bisa dinyatakan sebagai graf berarah tanpa siklus, **pakai Functional API, bukan subclassing**. Seluruh contoh di sisa buku memakai Functional API – tetapi dengan **lapis-lapis subclass** di dalamnya. Kombinasi itu memberi keluwesan pengembangan sekaligus keuntungan Functional API.

02

Lingkar pelatihan bawaan

Metrik kustom, callback, dan TensorBoard.

Menulis metrik sendiri

listing 7.18 – RMSE sebagai metrik kustom

PYTHON

```

from keras import ops

class RootMeanSquaredError(keras.metrics.Metric):
    def __init__(self, name="rmse", **kwargs):
        super().__init__(name=name, **kwargs)
        self.mse_sum = self.add_weight(name="mse_sum", initializer="zeros")
        self.total_samples = self.add_weight(name="total_samples", initializer="zeros")

    def update_state(self, y_true, y_pred, sample_weight=None):
        y_true = ops.one_hot(y_true, num_classes=ops.shape(y_pred)[1])
        self.mse_sum.assign_add(ops.sum(ops.square(y_true - y_pred)))
        self.total_samples.assign_add(ops.shape(y_pred)[0])

    def result(self):
        return ops.sqrt(self.mse_sum / self.total_samples)

    def reset_state(self):
        self.mse_sum.assign(0.)
        self.total_samples.assign(0.)

```

supaya objek metrik yang sama bisa dipakai
lintas-epoch dan lintas latih/evaluasi

Tiga metode itulah kontraknya: `update_state()` memperbarui, `result()` melaporkan nilai sekarang, `reset_state()` mengosongkan tanpa perlu membuat objek baru.

Callback: dari pesawat kertas jadi dron

Meluncurkan pelatihan pada data besar selama puluhan epoch dengan `model.fit()` itu sedikit seperti melempar pesawat kertas: setelah dorongan awal, Anda tidak punya kendali atas lintasannya maupun tempat mendaratnya. API callback Keras mengubah panggilan `model.fit()` Anda dari pesawat kertas menjadi **dron cerdas dan otonom** yang bisa memeriksa dirinya sendiri dan bertindak.

Chollet & Watson, bab 7.3.2

Model checkpointing

Menyimpan keadaan model di berbagai titik selama pelatihan.

Early stopping

Menghentikan pelatihan saat rugi validasi berhenti membaik – dan menyimpan model terbaik yang sudah diperoleh.

Menyetel parameter dinamis

Misalnya learning rate optimizer, di tengah pelatihan.

Mencatat & menggambar

Bilah kemajuan `fit()` yang sudah Anda kenal itu **sebenarnya sebuah callback**

callback bawaan (tidak lengkap)

```
keras.callbacks.ModelCheckpoint
keras.callbacks.EarlyStopping
keras.callbacks.LearningRateScheduler
keras.callbacks.ReduceLROnPlateau
keras.callbacks.CSVLogger
```

PYTHON

EarlyStopping + ModelCheckpoint: pasangan baku

Listing 7.19

PYTHON

```
callbacks_list = [  
    keras.callbacks.EarlyStopping(  
        monitor="accuracy",    # dipantau, jadi harus ada di metrics model  
        patience=1,           # berhenti bila tak membaik lebih dari satu epoch  
    ),  
    keras.callbacks.ModelCheckpoint(  
        filepath="checkpoint_path.keras",  
        monitor="val_loss",  
        save_best_only=True,    # berkas tidak ditimpa kecuali val_loss membaik  
    ),  
]  
  
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",  
              metrics=["accuracy"])  
model.fit(train_images, train_labels, epochs=10,  
        callbacks=callbacks_list,  
        validation_data=(val_images, val_labels)) # WAJIB, karena val_* dipantau
```

Ini menggantikan pola boros di bab 4-5: latih sampai overfit untuk **mencari** epoch terbaik, lalu latih ulang dari nol sebanyak itu. Dengan pasangan callback ini, **sekali jalan sudah cukup**

menyimpan dan memuat manual

PYTHON

```
model.save("my_checkpoint_path.keras")  
model = keras.models.load_model("checkpoint_path.keras")
```

Callback kustom – enam kait yang tersedia

kait yang bisa Anda isi

PYTHON

```
on_epoch_begin(epoch, logs)
on_epoch_end(epoch, logs)
on_batch_begin(batch, logs)
on_batch_end(batch, logs)
on_train_begin(logs)
on_train_end(logs)
```

Semuanya dipanggil dengan argumen `logs`, sebuah kamus berisi informasi tentang batch, epoch, atau jalannya pelatihan – metrik latih dan validasi.

listing 7.20 – riwayat rugi per batch

PYTHON

```
class LossHistory(keras.callbacks.Callback):
    def on_train_begin(self, logs):
        self.per_batch_losses = []

    def on_batch_end(self, batch, logs):
        self.per_batch_losses.append(
            logs.get("loss"))

    def on_epoch_end(self, epoch, logs):
        plt.clf()
        plt.plot(range(len(self.per_batch_losses)),
                 self.per_batch_losses)
        plt.savefig(f"plot_at_epoch_{epoch}", dpi=300)
        self.per_batch_losses = []
```


TensorBoard: memutar lingkaran kemajuan lebih cepat

Pantau metrik

Secara visual, selama pelatihan berjalan.

Gambar arsitektur

Visualisasi model.

Histogram

Aktivasi dan gradien.

Embedding 3D

Jelajahi ruang embedding.

memakainya

PYTHON

```
tensorboard = keras.callbacks.TensorBoard(log_dir="/full_path_to_your_log_dir")
model.fit(train_images, train_labels, epochs=10,
          validation_data=(val_images, val_labels),
          callbacks=[tensorboard])
```

menjalankan servernya

BASH

```
# di mesin lokal
tensorboard --logdir /full_path_to_your_log_dir

# di dalam notebook Colab
%load_ext tensorboard
%tensorboard --logdir /full_path_to_your_log_dir
```

03

Menulis lingkaran pelatihan sendiri

Saat fit() tidak cukup – dan cara tetap memakainya.

Kapan fit() memang tidak cukup

Generative learning

Tidak ada target eksplisit. Diperkenalkan di **bab 16**.

Self-supervised

Targetnya diambil **dari masukannya sendiri**.

Reinforcement learning

Pembelajaran didorong **imbalan sesekali** – seperti melatih anjing.

Dua kehalusan yang harus diperhatikan saat menulis lingkaran sendiri:

- 1 **training=True** **wajib diteruskan**. Lapis seperti `Dropout` berperilaku berbeda saat latih dan saat inferensi. `dropout(inputs, training=True)` menjatuhkan sebagian aktivasi; `training=False` tidak melakukan apa pun. Jadi: `predictions = model(inputs, training=True)`.
- 2 **Pakai `model.trainable_weights`, bukan `model.weights`**. Model punya dua jenis bobot: **terlatih** (diperbarui backpropagation) dan **tak terlatih** (diperbarui lapis pemiliknya saat lintasan maju). Di antara lapis bawaan Keras, satu-satunya yang punya bobot tak terlatih adalah `BatchNormalization` – ia melacak rerata dan simpangan baku data yang lewat (bab 9).

Satu langkah pelatihan, tiga backend

TensorFlow

PYTHON

```
def train_step(inputs, targets):
    with tf.GradientTape() as tape:
        predictions = model(inputs, training=True)
        loss = loss_fn(targets, predictions)
        gradients = tape.gradient(loss, model.trainable_weights)
        optimizer.apply(gradients, model.trainable_weights)
    return loss
```

PyTorch

PYTHON

```
def train_step(inputs, targets):
    predictions = model(inputs, training=True)
    loss = loss_fn(targets, predictions)
    loss.backward()           # isi nilai gradien
    gradients = [w.value.grad for w in model.trainable_weights]
    with torch.no_grad():     # WAJIB di dalam no_grad()
        optimizer.apply(gradients, model.trainable_weights)
    model.zero_grad()        # WAJIB - backward() itu menumpuk
    return loss
```

Pada PyTorch, `model.zero_grad()` **kritis**: panggilan `backward()` bersifat menambahkan. Kalau gradien tidak dikosongkan tiap langkah, nilainya menumpuk dan **pelatihan tidak akan berjalan**

JAX: paling rumit, karena tanpa keadaan sama sekali

stateless_call() dan value_and_grad

PYTHON

```
# lintasan maju tanpa keadaan
outputs, non_trainable_weights = model.stateless_call(
    trainable_weights, non_trainable_weights, inputs)

def compute_loss_and_updates(trainable_variables, non_trainable_variables,
                             inputs, targets):
    outputs, non_trainable_variables = model.stateless_call(
        trainable_variables, non_trainable_variables, inputs, training=True)
    loss = loss_fn(targets, outputs)
    return loss, non_trainable_variables      # skalar DULU, sisanya jadi 'aux'

# jax.grad hanya menerima fungsi yang mengembalikan skalar -> pakai has_aux
grad_fn = jax.value_and_grad(compute_loss_and_updates, has_aux=True)
(loss, non_trainable_weights), gradients = grad_fn(
    trainable_variables, non_trainable_variables, inputs, targets)

# optimizer pun punya keadaan (momentum dll), jadi ada padanan tanpa-keadaannya
trainable_variables, optimizer_variables = optimizer.stateless_apply(
    optimizer_variables, gradients, trainable_variables)
```

Pola yang sama berlaku untuk metrik: di JAX, metode yang mengubah keadaan seperti `update_state()` tidak bisa dipanggil di dalam fungsi tanpa keadaan. Padanannya: `stateless_update_state()`, `stateless_result()`, `stateless_reset_state()`

Memakai metrik di tingkat rendah

metrik biasa

PYTHON

```
metric = keras.metrics.SparseCategoricalAccuracy()
targets = ops.array([0, 1, 2])
predictions = ops.array([[1, 0, 0],
                          [0, 1, 0],
                          [0, 0, 1]])
metric.update_state(targets, predictions)
print(f"result: {metric.result():.2f}")
```

OUTPUT

result: 1.00

melacak rerata skalar

PYTHON

```
values = ops.array([0, 1, 2, 3, 4])
mean_tracker = keras.metrics.Mean()
for value in values:
    mean_tracker.update_state(value)
print(f"Mean: {mean_tracker.result():.2f}")
```

OUTPUT

Mean: 2.00

Jangan lupa `metric.reset_state()` saat mau mengosongkan hasilnya – **di awal tiap epoch pelatihan, dan di awal evaluasi**. Lupa melakukannya membuat angka metrik **tercampur antar-epoch**

Jalan tengah: train_step() sendiri, fit() tetap dipakai

listing 7.21 – versi TensorFlow

PYTHON

```

loss_fn = keras.losses.SparseCategoricalCrossentropy()
loss_tracker = keras.metrics.Mean(name="loss")

class CustomModel(keras.Model):
    def train_step(self, data):
        inputs, targets = data
        with tf.GradientTape() as tape:
            predictions = self(inputs, training=True) # self, bukan model
            loss = loss_fn(targets, predictions)
            gradients = tape.gradient(loss, self.trainable_weights)
            self.optimizer.apply(gradients, self.trainable_weights)
            loss_tracker.update_state(loss)
        return {"loss": loss_tracker.result()} # nama metrik -> nilainya

@property
def metrics(self):
    return [loss_tracker] # didaftarkan agar reset_state() dipanggil otomatis
                        # di awal tiap epoch dan di awal evaluate()

```

- ♦ Pola ini **tidak menghalangi** Anda memakai Functional API – berlaku untuk Sequential, Functional, maupun subclass.
- ♦ Anda **tidak perlu** dekorator `@tf.function` atau `@jax.jit`; **kerangkanya yang mengerjakan itu untuk Anda.**

Yang wajib terbawa dari bab 7

- 1 **Progressive disclosure of complexity** – satu spektrum alur kerja di atas API bersama `Layer` dan `Model`, bukan beberapa kerangka terpisah.
- 2 **Sequential** untuk tumpukan sederhana; **Functional API** untuk graf lapis – dan itu yang dipakai sisa buku ini; **subclassing** untuk yang tidak bisa dinyatakan sebagai graf.
- 3 Functional memberi **akses ke keterhubungan lapis**: `plot_model()` dan ekstraksi fitur. Subclassing **membuang keduanya**.
- 4 Campuran terbaik: **model Functional yang berisi lapis-lapis subclass**.
- 5 **Metrik kustom** = `update_state` / `result` / `reset_state`. **Callback** = enam kait `on_*`.
- 6 **EarlyStopping** + **ModelCheckpoint** menggantikan pola latih-ulang yang boros itu.
- 7 Menulis `train_step()` sendiri: ingat **`training=True`** dan **`trainable_weights`**; di PyTorch jangan lupa **`zero_grad()`**; di JAX semuanya lewat padanan **`stateless_*`**.

NOTEBOOK

[03_train_step_kustom_per_backend.ipynb](#)

BAB BERIKUT

[Bab 8 — Klasifikasi citra](#)